SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME:Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 35%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

*I **B Reddy Balu (192312499)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

iii. Analyze the impact of heuristic accuracy on both algorithms

iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search

ii. Analyze the challenges of partial observability and dynamic environments

iii. Describe how the rover builds and updates its internal model

iv. Compare online search with uninformed and informed search strategies

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation

ii. Explain how backtracking search works in this scenario

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency

iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem

ii. Analyze the computational challenges of large game trees

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

iv. Design an evaluation function and justify the factors considered

v. Recommend a strategy for balancing optimality and real-time performance

Question 1: AI Drones for Emergency Medicine Delivery in a Flood Zone

Scenario understanding. The drone must find a route from a base to a delivery point in a graph where nodes are locations or waypoints and edges are possible flight or road segments. Edge costs vary with terrain and weather, edges may become blocked at any time, and only partial real-time updates are available. The only guidance is a straight-line-distance heuristic h(n). The goal is minimum delay and minimum risk, under time pressure.

(i) Greedy Best-First Search (GBFS) in this scenario

GBFS expands the node with the lowest heuristic value, f(n) = h(n). The drone would always head to the waypoint that appears closest to the hospital in a straight line, ignoring the cost already incurred (g) and the actual cost of the next edge.

Strengths under uncertainty

- Very fast decisions: few nodes are expanded and the cost per decision is tiny, which suits the urgency.
- Low memory use, so it can run on an onboard computer.
- Easy to re-run when a new blockage is reported, because it only needs a quick look at the current neighbours.
- Tends to make steady progress towards the goal when the terrain is mostly open.

Weaknesses under uncertainty

- **Not optimal:** it ignores terrain and weather costs, so it may choose a short but stormy or dangerous path.
- **Easily trapped:** straight-line distance does not know about flooded rivers or collapsed roads. The drone may rush into a dead end and then have to backtrack.
- **Incomplete on graphs** without a closed list; it can loop between two nodes when blockages appear.
- **No risk awareness:** it cannot trade a slightly longer route for a much safer one.
- Its quality depends entirely on the heuristic, which is weakest exactly where floods have changed the map.

(ii) A* in the same situation

A* uses f(n) = g(n) + h(n), where g(n) is the actual accumulated cost from the start (distance adjusted by terrain, wind, rain and risk) and h(n) is the straight-line estimate of the remaining cost to the goal.

- **Balance:** g(n) keeps the search honest about the real cost already paid, so a node reached via a stormy segment looks expensive. h(n) pulls the search towards the goal so it does not explore everywhere like Dijkstra.
- Because straight-line distance (scaled by the minimum cost per unit distance) never overestimates, it is **admissible**, and it is also consistent. A* is therefore **complete and optimal**.
  - Variable costs are handled naturally by placing them in g(n). Risk can be added to the edge cost, for example cost = distance × weather factor + risk penalty.

- Blocked edges are simply removed (cost = ∞) and the search re-plans.
- **Drawbacks:** it expands more nodes and stores an open and closed list, so it is slower and uses more memory than GBFS in large maps.

(iii) Impact of heuristic accuracy

Heuristic situation	**Effect on Greedy Best-First**	**Effect on A***
Accurate (close to true cost)	Goes almost straight to the goal; fast and often near-optimal	Expands very few nodes; optimal
Underestimates (admissible, but weak)	Wanders more; may still be misled	Still optimal, but expands more nodes (approaches Dijkstra when h = 0)
Overestimates (inadmissible)	Behaves the same, since it uses only h; may over-avoid good areas	Can lose optimality and return a costlier or riskier route
Misleading (e.g. ignores flooded river)	Can send drone into dead ends repeatedly	g(n) corrects the error over time, so it recovers, but with extra expansions

In short, GBFS depends completely on h, so its behaviour degrades sharply with a poor heuristic. A* is robust because the real cost g(n) acts as a safety net; a good heuristic only affects its speed, provided it remains admissible.

(iv) Effect of dynamic changes (blocked paths)

- A previously computed path becomes invalid when an edge is blocked, so the drone must **re-plan**.
- **GBFS** re-plans cheaply but may repeat poor choices, oscillate near blockages, or fall into new dead ends.
- **A*** gives a good new route but recomputing from scratch is expensive when updates are frequent.
- Search effort grows with the number of blockages: more detours, more nodes expanded, and stale information may cause the drone to plan through edges that are already blocked.
- Partial updates mean the map is only partly known, so plans should be treated as provisional.
- **Remedies:** incremental or dynamic replanning algorithms (D* Lite, LPA*) that reuse earlier search effort, a receding-horizon approach (plan a short distance, execute, re-plan), and treating unknown edges optimistically but with a risk penalty.

(v) Recommendation

Recommended: A* (in its incremental form, D* Lite / weighted A*), using a risk-adjusted cost function.

- **Optimality:** minimum delay and risk is the explicit objective. A* with an admissible heuristic returns the cheapest route in terms of time and risk; GBFS gives no such guarantee.
- **Safety:** terrain, weather and risk penalties are built into g(n). GBFS cannot see these costs.

Real-time constraints: use Weighted A* (f = g + w·h with w slightly above 1) or an anytime version to return a good route quickly and refine it if time remains. Use D* Lite to repair the route when paths are blocked rather than start again.

Question 2: Smart City Traffic Signal Optimisation

Problem formulation as an optimisation problem

- **State / decision variables:** a complete signal plan for all N intersections: green time for each phase, cycle length and offset (start-time shift) relative to neighbours.
- **Objective function (minimise):** $F = w_1 \cdot (\text{Total Waiting Time}) + w_2 \cdot (\text{Fuel Consumption}) + w_3 \cdot (\text{Congestion/Queue Length})$, where the weights express city policy.
- **Constraints:** minimum and maximum green time per phase, minimum pedestrian crossing time, safety clearance (amber/all-red), maximum cycle length, and emergency vehicle priority.
- **Neighbourhood (moves):** change one green time by $\pm\delta$ seconds, shift an offset, or swap a phase order at one intersection.
- **Evaluation:** a traffic simulation or live sensor data gives the objective value for a candidate plan.

  Why traditional (uninformed / A*-style) search is inefficient:

  - The search space is enormous. If each of N = 300 intersections has k = 20 timing options, there are 20^{300} combinations.
  - There is no single goal state; the aim is to find the best-scoring configuration, so goal-based search has nothing to stop on.
  - The path to the solution is irrelevant, only the final configuration matters, so storing the paths (as BFS/DFS/A* do) wastes memory.
  - Traffic changes continuously, so a computed optimum quickly becomes outdated.
  - Evaluating each candidate is costly (it needs simulation), so exhaustive exploration is impossible.

  This is why **local search / metaheuristic** methods, which keep only the current solution and improve it, are appropriate.

(i) Hill Climbing and its limitations

Hill Climbing starts from the current signal plan, generates neighbouring plans (e.g. extend the green of one road by 5 seconds), evaluates them, and moves to the neighbour with the best improvement. It stops when no neighbour is better.

```
current = initial_plan() loop:     neighbour = best_neighbour(current)     if F(neighbour) >= F(current): return current   # stuck     current = neighbour
```

Advantages: simple, very low memory, fast, and ideal for continuous small adjustments as traffic changes.

Limitations: it only accepts improving moves, so it gets stuck at local optima; it cannot cross flat regions; its result depends heavily on the starting plan; it is greedy and has no mechanism to escape.

(ii) Local maxima, plateaus and ridges (real-world meaning)

Problem	Meaning in the algorithm	Real-world traffic interpretation
Local maximum (here, local minimum of cost)	A state better than all neighbours but worse than the global best	A plan that is good for one junction (long green on the main road) but any single change makes things worse, even though a very different plan would reduce delay city-wide
Plateau	A flat region where neighbouring states have equal value	Changing a green time by a few seconds makes no measurable difference to waiting time, so the algorithm has no direction to move in and may wander or stop
Ridge	A narrow path of improvement that needs several variables to change together	A 'green wave' on an arterial road needs the offsets of several adjacent signals to change at the same time. Changing one signal alone makes things worse, so single-move hill climbing cannot follow the ridge

(iii) How Simulated Annealing and Genetic Algorithms overcome these limitations

Simulated Annealing (SA):

- A random neighbour is generated. If it is better, accept it. If it is worse, accept it with probability e^(−ΔF/T), where ΔF is the increase in cost and T is the temperature.
- At high T the algorithm explores freely and can climb out of local optima and cross plateaus; as T is slowly reduced (cooling schedule) it becomes selective and settles on a good solution.
- It needs little memory and is easy to add to an existing hill-climbing controller.

Genetic Algorithms (GA):

- **Chromosome:** the list of green times and offsets for all intersections. **Fitness:** 1/F (lower waiting time, fuel and congestion gives higher fitness).
- **Selection** keeps the better plans, **crossover** combines parts of two good plans (e.g. the north-zone timings of one with the south-zone timings of another), and **mutation** randomly changes a few values.
- Because a whole population is searched in parallel, GA explores many regions at once, so it is much less likely to be trapped in a single local optimum, and crossover can change several variables together, which helps with ridges.
- Different zones of the city can be evolved in parallel.

Aspect	Simulated Annealing	Genetic Algorithm
Candidates held	One	Population
Escaping local optima	Probabilistic acceptance of worse	Diversity, crossover and mutation

Aspect	Simulated Annealing	Genetic Algorithm
	moves	
Compute cost	Low	Higher (many evaluations)
Parallelism	Limited	Natural
Suited for	Continuous online tuning	Global search / offline planning

(iv) Recommended approach

Recommendation: a hybrid (memetic) approach – Genetic Algorithm for global planning, combined with Simulated Annealing / hill-climbing for real-time adaptation.

- **Scalability:** divide the city into zones/corridors and run GA per zone in parallel, then coordinate offsets between zones. GA handles the very large search space and many variables.
- **Adaptability:** GA periodically (e.g. every few hours, or for different day types) produces good baseline plans. During the day, SA or hill climbing with small perturbations adjusts the timings as live traffic data arrives. Previous good solutions are used as the starting population so the search restarts quickly rather than from zero.
- **Robustness:** random restarts, SA's controlled acceptance of worse moves and population diversity together avoid local optima, plateaus and ridges.
- **Safety:** all candidate plans must satisfy the hard constraints (minimum green, pedestrian time, emergency priority) before being deployed.

Question 3: Autonomous Mars Rover – Online Search

(i) Why an online search agent is needed instead of offline search

- **Offline search** computes a complete plan before acting, which requires a full, accurate model (a complete map). The rover has no complete map of Mars.
- **Online search** interleaves planning and acting: the agent takes an action, observes the result, updates its knowledge and then chooses the next action. This is the only workable approach in unknown terrain.
- Communication delay with Earth (minutes) rules out remote step-by-step control, so decisions must be made on board.
- Plans made before exploring would often prove wrong because of hidden craters and rocks, so an offline plan would be unsafe or impossible to complete.
- In an unknown environment the agent can only expand the node it is physically at, so the search is local by nature (like online DFS or LRTA*).

(ii) Challenges of partial observability and dynamic environments

- **Limited sensing range:** cameras and LIDAR see only nearby terrain, and shadows, dust and slopes hide hazards.
- **Sensor noise and uncertainty:** a shadow may look like a crater, or a crater may be missed. The rover cannot be sure which state it is actually in.
- **Localisation errors:** wheel slip and drift mean the rover's belief about its position can be wrong.
- **Unknown consequences of actions:** soft sand may trap the wheels. The true cost of actions is only discovered by trying them.
- **Dynamic conditions:** dust storms, changing light, battery and temperature limits, and moving sand all change what is safe.
- **Irreversible actions:** driving into a crater may be unrecoverable, so exploration must be cautious.
- **Exploration vs exploitation:** the rover must balance exploring unknown areas (to find samples and a route) against safely using the known map.

(iii) How the rover builds and updates its internal model

- **Initial model:** an empty or low-resolution map (e.g. an occupancy grid where every cell is 'unknown') plus the start position and goal/sample locations.
- **Sense:** cameras, LIDAR, IMU and wheel odometry detect obstacles, slope and surface type.
- **Update:** each cell is marked free, obstacle, hazardous or unknown, often with a probability (Bayesian update, e.g. Pr(occupied) is increased when the sensor returns a hit). Position is estimated with SLAM (simultaneous localisation and mapping) and filters such as a Kalman or particle filter.
- **Store cost information:** visited cells, traversal difficulty, and learned heuristic values (as in LRTA*, where h(s) is updated after each move: h(s) = min over actions of [c(s,a) + h(s')]).
    - **Replan:** after each update, if the planned path is now blocked or has become risky, the rover re-plans from its current position.

- The map thus grows incrementally and the **belief state** (what the rover thinks is true) is continually refined.

(iv) Online search compared with uninformed and informed search

Feature	**Uninformed (BFS, DFS, UCS)**	**Informed (Greedy, A*)**	**Online search (online DFS, LRTA*, D* Lite)**
Knowledge needed	Complete model of the state space	Complete model plus a heuristic	No complete model; learns while moving
When it plans	Before acting	Before acting	While acting (interleaved)
Use of information	None beyond the goal test	Heuristic guides expansion	Local sensing plus heuristics that are updated by experience
Handles unknown terrain?	No	No (needs the map)	Yes
Backtracking	Virtual (in memory)	Virtual (in memory)	Physical, and can be costly or unsafe
Optimality	BFS/UCS yes for their cost measure	A* yes with admissible h	Not guaranteed; the quality improves with learning
Suitability for Mars rover	Poor	Fair only for the known region	Best

(v) Most suitable approach and justification

Recommended: an online informed search – LRTA* / D* Lite combined with a sensor-based map update and safety-weighted costs.

- **Uncertainty:** the rover plans only in the region it currently knows and treats unknown cells with an optimistic cost plus a risk margin. It keeps a probabilistic map and re-plans whenever new observations arrive.
- **Adaptability:** LRTA* learns and improves its heuristic values from experience, and D* Lite repairs its plan efficiently when the map changes, instead of replanning from scratch.
- **Safety:** hazard cells (craters, steep slopes) are given very high or infinite cost and inflated by a safety margin; the rover moves in short, verifiable steps, always keeping an escape route and a way back to a safe state, and stops to rescan when confidence is low.
- **Efficiency:** low computation and memory per step suit limited onboard power and processing.
- **Conclusion:** the problem is unknown, partially observable and dynamic, so only an online, informed, safety-aware search can balance exploration with survival.

Question 4: University Exam Timetabling as a CSP

Formulation as a Constraint Satisfaction Problem

A CSP is defined by a set of variables X, a domain D for each variable and a set of constraints C. The goal is to find a complete assignment that satisfies every constraint.

(i) Variables, domains and constraints

Variables: one variable for each exam (course) E_i, i = 1…n. Each variable has two parts: the time slot T_i and the hall H_i (or, equivalently, one variable with value (slot, hall)).

Domains:

- Dom(T_i) = {all available exam slots}, e.g. Day1-Morning, Day1-Afternoon, … Day10-Afternoon.
- Dom(H_i) = {Hall A, Hall B, …} limited to halls whose capacity ≥ number of students enrolled in E_i (a unary constraint that shrinks the domain).
- Domains are further reduced by faculty availability and special accommodation needs.

Constraints:

Requirement	**Constraint type**	**Formal expression**
No student has overlapping exams	Binary	If courses E_i and E_j share at least one student, then $T_i \neq T_j$
Limited exam halls	Global / binary	Two exams in the same slot cannot use the same hall: if $T_i = T_j$ then $H_i \neq H_j$. Number of exams per slot ≤ number of halls; hall capacity ≥ enrolment
Some subjects before others (prerequisite order)	Binary (precedence)	If A must come before B, then T_a < T_b
Faculty availability	Unary and binary	$T_i \in$ available slots of the invigilator/examiner; one faculty member cannot supervise two exams at the same time
Special accommodations	Unary	For students needing extra time, a ground-floor or separate room: $H_i \in$ accessible halls, and a longer slot or a buffer slot is reserved

The overlap constraints form a **conflict graph** (nodes = exams, edge = shared students), so the problem resembles **graph colouring** (colours = time slots) with extra constraints. All constraints above are hard constraints. Preferences (e.g. avoiding three exams in a row) can be modelled as soft constraints.

(ii) How backtracking search works

Backtracking is depth-first search that assigns one variable at a time and checks consistency immediately.

```
BACKTRACK(assignment):  if assignment complete: return assignment  var = SELECT-UNASSIGNED-VARIABLE()  for value in ORDER-DOMAIN-VALUES(var):      if value consistent with assignment:
```

```
add (var = value);  result = BACKTRACK(assignment)        if result != failure: return result        remove
(var = value)     # backtrack   return failure
```

Example: assign Maths → Day1-Morning, Hall A. Next, Physics shares students with Maths, so it cannot use Day1-Morning and gets Day1-Afternoon. If later an exam has no legal slot or hall left, the algorithm returns to the most recent decision, changes it (e.g. moves Physics to Day2), and continues. Because assignments are checked at once, whole branches of invalid timetables are never explored. Without improvements, however, the worst case is exponential.

Heuristics to speed it up: MRV (choose the exam with the fewest legal slots), degree heuristic (choose the exam with the most conflicts, such as a large first-year core subject) and LCV (try the value that rules out the fewest options for neighbours).

(iii) Constraint propagation and forward checking

- **Forward checking:** after assigning E_i = (slot, hall), remove inconsistent values from the domains of its unassigned neighbours. For example, once Maths takes Day1-Morning, that slot is deleted from the domain of every course that shares students with Maths, and Hall A is removed from other exams in that slot.
- **Early failure detection:** if any neighbour's domain becomes empty, the algorithm backtracks immediately instead of discovering the dead end much deeper in the tree. This avoids huge amounts of wasted search.
- **Stronger propagation (arc consistency, AC-3 / MAC):** also removes values that cannot be part of any solution given the constraints between unassigned variables, e.g. if A must precede B and B has only the earliest slot left, A has no valid slot, so the conflict is found at once.
- Combined with MRV, forward checking leads to the most-constrained exams being tackled first and typically reduces the number of nodes explored by orders of magnitude.

(iv) Real-world challenges and the best strategy for scalability

Challenges

- Size: thousands of students and hundreds of courses give a huge search space and a dense conflict graph.
- The problem may be over-constrained (no timetable satisfies everything), so some soft constraints must be relaxed.
- Late changes: withdrawals, new accommodations, hall unavailability and faculty changes require re-timetabling with minimal disruption.
- Fairness and quality: spreading exams so students do not face too many exams in succession, a goal that is not captured by hard constraints.
- Plain backtracking can thrash (repeatedly failing for the same reason).

Recommended strategy: a hybrid CSP approach.

- Model the problem as a CSP and use **backtracking with MRV + degree heuristic + forward checking / MAC (AC-3)** to build a feasible timetable.

Question 5: Strategic Game AI – Minimax and Alpha-Beta Pruning

(i) How Minimax models the problem

Minimax models a two-player, zero-sum, adversarial game with perfect information. Here, the AI is **MAX** and the human player is **MIN**.

- **States:** snapshots of the game (unit positions, resources, buildings, health, map control).
- **Actions:** legal moves for the player to act (build, attack, move, research). Real-time play is handled by discretising time into small decision steps.
- **Transition model:** the result of applying an action.
- **Terminal test / cut-off:** a won or lost position, or the depth limit.
- **Utility / evaluation:** a numeric score from the AI's point of view (positive is good for the AI).

MAX chooses the move that maximises its value, assuming MIN replies with the move that minimises it: Minimax(s) = max over actions of Minimax(result(s,a)) at MAX nodes, and min at MIN nodes. The AI therefore plays the move that is best against the strongest possible human reply, i.e. the best worst-case result.

(ii) Computational challenges of large game trees

- Time complexity is $\mathbf{O(b^d)}$ where b is the branching factor and d the search depth. With b = 30 and d = 10, there are about $30^{10} \approx 5.9 \times 10^{14}$ positions, far too many to examine within a time limit.
- Space complexity is O(b·d) for depth-first Minimax, which is manageable, but the time is not.
- Real-time strategy games have a very high branching factor because many units can act at once, and the game is continuous, with hidden information and randomness, so the tree is even larger and only approximate.
- Strict response-time limits (milliseconds) mean the AI cannot search to the end of the game and must stop at a depth limit and rely on an evaluation function.
- Evaluating each state is itself expensive; repeated states (transpositions) waste effort; and the horizon effect (a disaster just beyond the search depth) can mislead the AI.

(iii) How Alpha-Beta pruning improves efficiency

Alpha-Beta is Minimax with two bounds passed down the tree: $\boldsymbol{\alpha}$ = the best value MAX can already guarantee along the current path (starts at $-\infty$), and $\boldsymbol{\beta}$ = the best value MIN can already guarantee (starts at $+\infty$). A branch is cut off as soon as $\boldsymbol{\alpha \geq \beta}$, because the opponent would never allow the game to reach that branch.

Detailed reasoning with an example: suppose MAX at the root has found that move A guarantees a value of 6 ($\alpha = 6$). While examining move B, the first reply MIN considers gives a value of 3. MIN would pick a reply of at most 3 for move B, so B is worth at most 3, which is worse than 6. There is no need to examine MIN's other replies to B; they are pruned ($\beta = 3 \leq \alpha = 6$). The final decision is identical to full

Minimax.

- **Correctness:** pruning never changes the Minimax result; it only skips branches that cannot influence the decision.
- **Best case (perfect move ordering):** about O(b^(d/2)). In practice this lets the AI search **twice as deep** in the same time. For b = 30, d = 10: $30^5 \approx 2.4 \times 10^7$ nodes instead of 5.9×10^{14}.
- **Worst case:** $O(b^d)$, when the best moves are examined last and nothing is pruned.
- **Move ordering is the key:** examine promising moves first (captures, attacks, moves from the previous iteration, killer moves). Good ordering gets close to the best case.

(iv) Evaluation function design

Since the tree cannot be searched to the end, a heuristic **evaluation function** estimates the value of a non-terminal state. A weighted linear form is simple and quick to compute:

```
Eval(s) = w1·Army + w2·Economy + w3·Tech + w4·MapControl + w5·BaseSafety + w6·Mobility + w7·Production
```

Factor	What it measures	Why it is included
Army strength (difference)	Total health × attack power of my units minus the opponent's	Direct measure of the ability to win fights
Economy / resources	Resource income rate, stockpile, number of workers	Long-term strength; a stronger economy converts into a stronger army
Technology / upgrades	Researched upgrades and unlocked unit types	Quality of the units and future options
Map control / vision	Territory, key chokepoints and resource sites controlled	Gives resources and tactical advantage, and denies them to the enemy
Base safety / structure health	Health of command buildings and defences; units under threat	Losing the main base usually means losing the game
Production capacity / mobility	Number of production buildings, unit mobility, available moves	Ability to reinforce quickly and keep options open

Justification: the factors are cheap to compute, correlate with winning, and capture both short-term (army, safety) and long-term (economy, technology) aspects. The weights can be tuned by self-play or learning. The function should be symmetric (positive when favouring the AI, negative for the human) and should give very large values for a win or loss. Weights may be adjusted by game phase (early game: economy; late game: army and base safety).

(v) Strategy for balancing optimality and real-time performance

Recommended: depth-limited Alpha-Beta with iterative deepening, strong move ordering and a good evaluation function, within a fixed time budget.

Summary of Recommendations

Q	Scenario	Recommended approach	Key reason
1	Flood-zone drones	A* (weighted / D* Lite) with risk-aware cost	Optimal, safe, can replan when paths are blocked
2	Traffic signals	Hybrid: Genetic Algorithm + SA / hill climbing	Scales to huge space, escapes local optima, adapts online
3	Mars rover	Online informed search (LRTA* / D* Lite)	Unknown map, partial observability, safety
4	Exam timetable	CSP: backtracking + MRV + forward checking / MAC + min-conflicts repair	Guarantees hard constraints, scalable, fast repair
5	Strategy game AI	Iterative-deepening Alpha-Beta with evaluation function	Deep search within strict time limits

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understandin g of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned , logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand